Fatih Jawwad Al Mumtaz[1] **Amirul Al Aziz**[2]

[1]Department of Informatics [2]Dept. of Agricultural Industrial Technology
Faculty of Science and Technology, Universitas Darussalam Gontor, Ponorogo, Indonesia
fatihalmumtaz76@student.cs.unida.gontor.ac.id amirulalaziz57@student.tip.unida.gontor.ac.id
Zenodo DOI: 10.5281/zenodo.23765521 | **License: GNU AGPLv3**

ICAITech 2026

Context & Motivation: Always-On Voice at the Extreme Edge

The Edge Keyword Spotting (KWS) Workload

- **Always-On Listening**: Continuously scans streaming audio for trigger keywords (e.g., wake-words).
- **Canonical TinyML Benchmark**: Standard benchmark in the MLPerf Tiny suite (Banbury et al.).
- **Hard Constraints**: Ultra-low latency (< 1 ms) and strictly bounded energy (mW).

Why On-Device Inference? (SDG 3 Alignment)

- **No Cloud Dependency**: Operates robustly in rural clinics and low-connectivity environments.
- **Strict Audio Privacy**: Voice and acoustic health data never leave the edge device.
- **Target 3.8**: Low-cost voice assistive tech for elderly care and hands-free diagnostics.

The Problem: Habitual Deployment Assumptions

TinyML literature routinely treats **TensorFlow Lite (TFLite)** as the default and fastest runtime on Arm processors. But on modern 64-bit Arm SBCs (like Cortex-A76 on Pi 5), **is TFLite universally faster than ONNX Runtime across diverse architectures?**

Benchmark Model Zoo & Front-End Standardization

All models consume an identical $49 \times 10 \times 1$ **MFCC frame** and output a **12-way softmax** (10 commands + silence + unknown).

Model	Parameters	Structural Summary
DS-CNN (MLPerf Ref.)	22,604	10×4 Conv (stride 2) + $4 \times$ (3×3 Depthwise + 1×1 Pointwise) + GAP + Dense
DNN (Baseline)	88,396	4-layer MLP: 490–128–128–64–12 (lower-bound anchor)
LSTM	19,980	Unrolled LSTM(64) over 49 time-steps + Dense head
CRNN	56,844	3×3 Conv$\times 2$ + MaxPool$\times 2$ + Unrolled GRU(64) + Dense

True Float32 DS-CNN Reconstruction

The official MLPerf artifact is actually **mixed-precision** (int8 conv, fp32 remainder). We decompiled and reconstructed a true-fp32 Keras model to guarantee 100% fair export to ONNX and TFLite.

Full INT8 Post-Training Quantization

Re-quantized with 1,000 representative clips (seed 42) into **true full-int8** flatbuffers (float I/O, int8 internal GEMM/conv operators).

Controlled Benchmark Protocol on Raspberry Pi 5

Hardware & OS Environment

- **SoC**: Broadcom BCM2712 (Raspberry Pi 5)
- **Cores**: Quad-core Arm Cortex-A76 @ 2.4 GHz
- **RAM**: 8 GB LPDDR4X (synchronous)
- **OS**: Debian 64-bit GNU/Linux (Kernel 6.18)
- **Governor**: Pinned to `performance` (removes dynamic clock throttling confounds)
- **Runtimes**: LiteRT (`ai-edge-litert 2.1.6`) vs `onnxruntime 1.28`

Rigorous Statistical Protocol

- **Warmup**: 1,000 consecutive unmeasured runs
- **Measurement**: 1,000 timed passes sequentially (never interleaved)
- **Outlier Removal**: Filtered using 1.5 $\times$ IQR rule
- **Confidence Intervals**: 10,000-resample bootstrap (95% CI)
- **Dual Test Sets (Speech Commands v2)**:
  1. **10-Command Subset**: 4,074 official test clips
  2. **Full 12-Class Task**: 12,005 clips (including 6,931 unknown + 1,000 background silence)

Accuracy Evaluation: 10-Command vs Full 12-Class Task

	10-Command Subset (4,074 clips)			Full 12-Class Task (12,005 clips)		
Model	**fp32 (%)**	**int8 (%)**	**Δ (pts)**	**fp32 (%)**	**int8 (%)**	**Δ (pts)**
DS-CNN (ref.)	90.55	90.57	**+0.02**	95.49	95.21	**−0.28**
DNN (anchor)	79.63	79.65	**+0.02**	72.65	71.90	**−0.75**
LSTM	90.48	87.43	**−3.05**	88.84	84.72	**−4.12**
CRNN	**92.56**	**92.17**	**−0.39**	**92.08**	**91.60**	**−0.48**

1. Numerical Parity

TFLite fp32 and ONNX fp32 produce **identical top-1 predictions** across all 12,005 clips (max absolute diff $< 3 \times 10^{-6}$).

2. Recurrent PTQ Cost

LSTM suffers the largest drop (**−3.05 pts** on subset, **−4.12 pts** on full task), proving recurrence is hardest to quantize without QAT.

3. DS-CNN & CRNN Robust

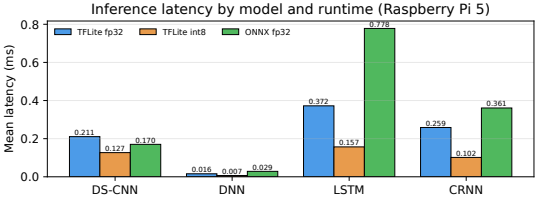
Full-int8 quantization has negligible accuracy cost on DS-CNN (−0.28 pts) and CRNN (−0.48 pts) on the full task.

Latency Benchmark: The Architecture Reversal

Model	TFLite fp32	TFLite int8	ONNX fp32	Ratio
DS-CNN	0.2107	**0.1268**	**0.1702**	**0.81** (ONNX)
DNN	**0.0158**	**0.0067**	0.0286	**1.81** (TFLite)
LSTM	**0.3722**	**0.1569**	0.7782	**2.09** (TFLite)
CRNN	**0.2586**	**0.1016**	0.3609	**1.40** (TFLite)

Key Architectural Insight

- **Why ONNX wins on DS-CNN**: DS-CNN is operator-dispatch bound (stride-2 conv, 4 depthwise blocks). ONNX's aggressive **kernel fusion** cuts operator dispatch overhead.
- **Why TFLite wins on the rest**: DNN, LSTM, and CRNN are dominated by large GEMM and sequential dataflow. TFLite's lightweight session machinery wins by **1.40–2.09**×.



Full INT8 Acceleration

Full-int8 DS-CNN runs in **0.1268 ms** (25% faster than ONNX fp32 and original mixed-precision artifact).

Thread-Sensitivity Sweep (T = 1, 2, 4 Cores)

Model & Runtime	T=1	T=2	T=4
DS-CNN TFLite	0.2119	0.1229	**0.0782**
DS-CNN ONNX	0.4379	0.2594	0.1689
Ratio (ONNX / TFLite)	**2.07**×	**2.11**×	**2.16**×
CRNN TFLite	0.2598	**0.1873**	0.2195
CRNN ONNX	0.5075	0.4186	0.3610
Ratio (ONNX / TFLite)	**1.95**×	**2.24**×	**1.64**×

Discovery: The Default Threading Confound

In default configurations, ONNX silently spawned a **4-thread threadpool**, while TFLite defaulted to **1 thread**!

Controlled Iso-Thread Findings

- **TFLite Wins at Every Thread Count**: When both runtimes are explicitly pinned to $T = 1$, TFLite is actually **2.07× faster** than ONNX on DS-CNN (0.2119 vs 0.4379 ms).
- **Scaling Diminishing Returns**: CRNN under TFLite is slower at 4 threads (0.2195 ms) than at 2 threads (0.1873 ms) due to thread synchronization overhead on small models (< 60k params).
- **Stable Ranking**: The runtime ranking is fully preserved across core sweeps.

On-Board SoC Energy Profiling via PMIC ADC

Model	Runtime	P_Δ (W)	t_{lat} (ms)	Energy E (mJ)
DS-CNN	**TFLite fp32**	1.59	0.217	**0.34 mJ**
DS-CNN	ONNX fp32	1.18	0.448	0.53 mJ
CRNN	**TFLite fp32**	1.85	0.274	**0.51 mJ**
CRNN	ONNX fp32	1.58	0.494	0.78 mJ
LSTM	**TFLite fp32**	1.81	0.406	**0.74 mJ**
LSTM	ONNX fp32	1.31	0.725	0.95 mJ

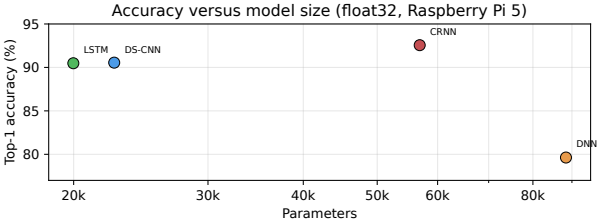
Direct PMIC ADC Telemetry

Captured via `vcgencmd pmic_read_adc` across 12 voltage rails. Measures pure SoC power, excluding peripherals.

The Power vs Energy Paradox

- **Power Draw**: TFLite draws **more instantaneous power** (+35% on DS-CNN: 1.59 W vs 1.18 W).
- **Total Energy**: Because TFLite executes more than 2× faster, it consumes **35% less energy** per inference (0.34 mJ vs 0.53 mJ)!
- **Takeaway**: For periodic and always-on edge audio, **faster completion to idle (Race-to-Sleep)** beats lower instantaneous power draw.

Accuracy vs Model Size Pareto Frontier



Architecture Trade-offs

- **CRNN (Best Accuracy)**: Reaches the highest accuracy (**92.56%** on 10-cmd, **92.08%** on full task) at 56.8k parameters.
- **LSTM (Parameter-Efficient)**: Reaches 90.48% with only **19,980 parameters**, but harder to quantize.
- **DS-CNN (Sweet Spot)**: Exceptional balance: 22.6k params, 95.49% full-task accuracy, lowest latency (0.1268 ms in int8).
- **DNN (Anchor)**: Spends 88.4k params for lowest accuracy (72.65% on 12-class).

Conclusion: Guidelines for Edge AI Practitioners

Core Findings

1. **Runtime Gap Does Not Generalize**: Defaulting to ONNX Runtime only pays off for convolution-heavy graphs like DS-CNN under multi-threading. For dense, recurrent, and hybrid models, **TensorFlow Lite is 1.40–2.09$\times$ faster**.
2. **Iso-Thread Parity Favors TFLite**: When pinned to equal thread counts ($T = 1, 2, 4$), TensorFlow Lite is faster for all architectures.
3. **Quantization vs Runtime Exchange Rate**: Full-int8 quantization provides a 1.7–2.5$\times$ speedup at negligible accuracy cost, except on recurrent units (LSTM).
4. **Race-to-Sleep Energy Advantage**: Lower inference latency directly translates to 35% lower battery energy consumption.

Actionable Recommendation

Choose your edge runtime based on architectural operator distribution, not convention. Test both TFLite and ONNX with pinned threads before production deployment.

Thank You for Your Attention!

Questions & Comments Welcome

Open Science & Permanent Artifact

- **Zenodo DOI**: `10.5281/zenodo.23765521`
- **GitHub Repository**: `github.com/jaweed3/kws-pi5-benchmark`
- **Open License**: GNU AGPLv3

Contact Authors

- **Fatih Jawwad Al Mumtaz**
  `fatihalmumtaz76@student.cs.unida.gontor.ac.id`
- **Amirul Al Aziz**
  `amirulalaziz57@student.tip.unida.gontor.ac.id`
- Faculty of Science and Technology,
  Universitas Darussalam Gontor, Indonesia